

🔗 Logistic Regression — Titanic Survival Prediction

Python 3.13 | scikit-learn | Status: Complete | Author: Sambit7848

Dataset: Titanic passenger data (built into the Seaborn library) **Goal:** Predict whether a passenger *survived* (1) or *did not survive* (0) the Titanic disaster.

This notebook is written as a **study-note style walkthrough** of Logistic Regression — each step includes a short explanation of *what* is being done and *why*, alongside the working code and its output.

📋 What this notebook covers

Section	Topic
1	Loading & cleaning the dataset
2	Univariate Logistic Regression (1 feature)
3	Multivariate Logistic Regression (5 features)
4	Comparing the two models & key takeaways

🧠 Note: What is Logistic Regression?

Logistic Regression is a **classification algorithm** used to predict a category (not a continuous number). Despite the name "regression," it is used when the target variable is discrete — most commonly **binary** (e.g., survived / did not survive, spam / not spam).

How it works, in short:

- A normal linear equation $z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b$ is calculated first (just like linear regression).
- That value z is then passed through the **sigmoid function**:

$$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$$

- The sigmoid squashes any real number into a range between **0 and 1**, which is interpreted as a **probability**.
- If the probability is ≥ 0.5 , the model predicts class **1** ; otherwise it predicts class **0** .

Univariate vs. Multivariate:

- **Univariate** Logistic Regression uses **one** input feature to predict the target.
- **Multivariate** Logistic Regression uses **two or more** input features, usually giving the model more context and better accuracy.

📁 Section 1: Load and Prepare the Dataset

Q1. Import libraries and load the dataset

Note: We import `pandas / numpy` for data handling, `seaborn` for the built-in Titanic dataset and plotting styles, and `matplotlib.pyplot` for chart display. `seaborn.load_dataset("titanic")` pulls a ready-made version of the famous Titanic passenger dataset.

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("titanic")
df.head()
```

Out[1]:	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton

Q2. Inspect shape, data types, and missing values

Note: Before modeling, we always profile the data:

- `.shape` tells us (rows, columns).
- `.dtypes` tells us whether a column is numeric (`int64` / `float64`) or categorical (`object` / `category` / `bool`).
- `.isnull().sum()` counts missing values per column — these must be handled before training a model, since scikit-learn cannot work with `NaN` values.

```
In [2]: print("Shape:", df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())
```

Shape: (891, 15)

Data Types:

```
survived      int64
pclass        int64
sex           str
age           float64
sibsp         int64
parch         int64
fare          float64
embarked      str
class         category
who           str
adult_male    bool
deck          category
embark_town   str
alive         str
alone         bool
dtype: object
```

Missing Values:

```
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64
```

Q3. Handle missing values

Note: How we handle missing data depends on the column:

- `deck` has too many missing values to be useful, so it is **dropped** entirely.
- `age` is numeric, so its missing values are filled with the **median** (robust to outliers like very old/young passengers).
- `embarked` / `embark_town` are categorical, so missing values are filled with the **mode** (most frequent category).

```
In [3]: df = df.drop(columns=["deck"])

df["age"] = df["age"].fillna(df["age"].median())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df["embark_town"] = df["embark_town"].fillna(df["embark_town"].mode()[0])

df.isnull().sum()
```

```
Out[3]: survived      0
        pclass        0
        sex           0
        age           0
        sibsp         0
        parch         0
        fare          0
        embarked      0
        class         0
        who           0
        adult_male     0
        embark_town    0
        alive          0
        alone          0
        dtype: int64
```

Q4. Identify and confirm the target variable

Note: The **target variable** is the column we are trying to predict — here, `survived`. Logistic Regression (in its basic form) needs a **binary** target, so we confirm it only contains the values `0` (did not survive) and `1` (survived).

```
In [4]: target = df["survived"]

print("Unique Values:", target.unique())
print("Is Binary:", set(target.unique()) == {0, 1})
```

```
Unique Values: [0 1]
Is Binary: True
```

Section 2: Univariate Logistic Regression

Note: In this section we build the simplest possible logistic model — using just **one feature**, `fare` — to predict `survived`. This helps establish a baseline before adding more features in Section 3.

Q5. Define X (features) and y (target)

Note: By convention, `X` (capital) holds the input feature(s) as a 2D DataFrame, and `y` (lowercase) holds the target as a 1D Series. Here `X` contains only the `fare` column.

```
In [5]: X = df[["fare"]]
        y = df["survived"]
```

Q6. Split into training and testing sets

Note: We never train and evaluate a model on the same data — that would overstate its performance. `train_test_split` randomly divides the data:

- `test_size=0.2` → 80% of rows go to training, 20% are held back for testing.
- `random_state=42` → fixes the random shuffle so results are reproducible.

```
In [6]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Q7. Build and train the model

Note: `LogisticRegression()` creates an untrained model object. Calling `.fit(X_train, y_train)` is the **training step** — the algorithm learns the coefficient(s) and intercept that best separate survivors from non-survivors based on `fare`.

```
In [7]: from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

▼ LogisticRegression ⓘ ?

► Parameters

Q8. Predict on the test set

Note: `.predict(X_test)` runs the trained model on unseen data and returns the predicted class (0 or 1) for each test row.

```
y_pred = model.predict(X_test)
print(y_pred)
```

[illegible]

Q9. Calculate accuracy

Note: Accuracy = (number of correct predictions) / (total predictions). It's the simplest way to evaluate a classifier, though it can be misleading on imbalanced datasets (not a major issue here).

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Accuracy: 0.6536312849162011

Q10. Coefficient and intercept

Note:

- The **coefficient** shows how much the log-odds of survival change for each one-unit increase in `fare` .
A positive coefficient means higher fare is associated with a higher chance of survival.
- The **intercept** is the log-odds of survival when `fare = 0` .

```
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
```

Coefficient: `[[0.01354448]]`

Intercept: [-0.93002768]

Q11. Confusion Matrix

Note: A **confusion matrix** breaks predictions into 4 categories:

- **True Negative (TN):** predicted "did not survive," actually didn't.
- **False Positive (FP):** predicted "survived," actually didn't.
- **False Negative (FN):** predicted "did not survive," actually did.
- **True Positive (TP):** predicted "survived," actually did.

It gives a more detailed picture of performance than accuracy alone — especially which *type* of mistake the model makes more often.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)

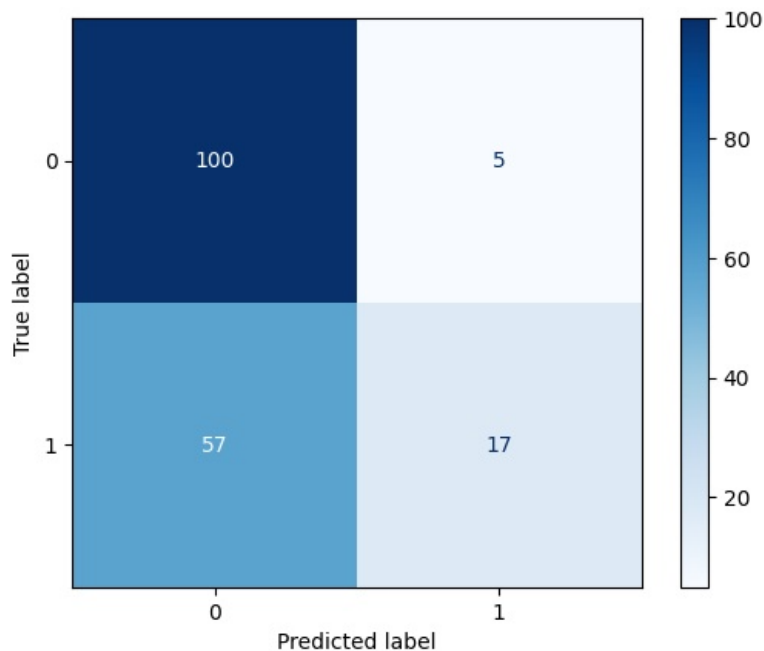
ConfusionMatrixDisplay(confusion_matrix=cm).plot(cmap="Blues")

plt.show()
```

```
cm = confusion matrix(y_test, y_pred)
```

```
ConfusionMatrixDisplay(confusion_matrix=cm).plot(cmap="Blues")
```

```
plt.show()
```



Section 3: Multivariate Logistic Regression

Note: Here we use **five features** — `pclass`, `age`, `fare`, `sibsp`, and `sex` — instead of just one. More relevant features usually give the model more signal to work with, which typically (though not always) improves accuracy.

Q12. Encode categorical column(s)

Note: Machine learning models need numbers, not text. `sex` is categorical ("male" / "female"), so we **encode** it into numbers using `.map()`: `male` → 0, `female` → 1. This is a simple form of **label encoding**.

```
In [12]: df["sex"] = df["sex"].map({"male": 0, "female": 1})
```

Q13. Define X and y with multiple features

Note: `X` now holds a DataFrame with 5 columns instead of 1 — this is what makes the model *multivariate*.

```
In [13]: X = df[["pclass", "age", "fare", "sibsp", "sex"]]
y = df["survived"]
```

Q14. Split into training and testing sets

Note: Same 80/20 split and random seed as Section 2, so the two models can be fairly compared.

```
In [14]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Q15. Build and train the model

Note: `max_iter=1000` raises the maximum number of optimization iterations. With more features, the solver sometimes needs extra iterations to **converge** (find the best-fit coefficients) without throwing a convergence warning.

```
In [15]: from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

```
Out[15]: ▼ LogisticRegression ⓘ ?
          ► Parameters
```

Q16. Predict on the test set

```
In [16]: y_pred = model.predict(X_test)

print(y_pred)
```

```
[0 0 0 1 1 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 1 0 0 0
 1 1 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 0 1 1 1 0 1 1 0 0 1 0 0 0 1 1 1 0 1
 0 0 1 1 1 1 0 1 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1
 0 1 0 1 0 0 0 1 0 0 1 1 0 0 0 0 1 1 1 0 1 0 0 1 0 1 1 0 0 1 0 1 0 0 1 1 0 0
 1 0 0 0 0 1 0 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 1 1 0 1 0 0 0 1 1 0 0 1 1 0 0]
```

Q17. Calculate accuracy

```
In [17]: from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Accuracy: 0.7932960893854749

Q18. Coefficients and the strongest feature

Note: With multiple features, `model.coef_` returns one coefficient per feature. Comparing their **absolute values** tells us which feature has the strongest pull on the prediction — the largest magnitude coefficient (positive or negative) is the most influential feature.

```
In [18]: feature_names = X.columns

print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)

coef_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": model.coef_[0]
})

print(coef_df)

strongest_feature = coef_df.loc[coef_df["Coefficient"].abs().idxmax()]

print("\nStrongest Feature:")
print(strongest_feature)
```

Coefficients: [[-0.94088044 -0.03091617 0.00290889 -0.3458014 2.56889891]]

Intercept: [1.63651227]

	Feature	Coefficient
0	pclass	-0.940880
1	age	-0.030916
2	fare	0.002909
3	sibsp	-0.345801
4	sex	2.568899

Strongest Feature:

Feature	sex
Coefficient	2.568899

Name: 4, dtype: object

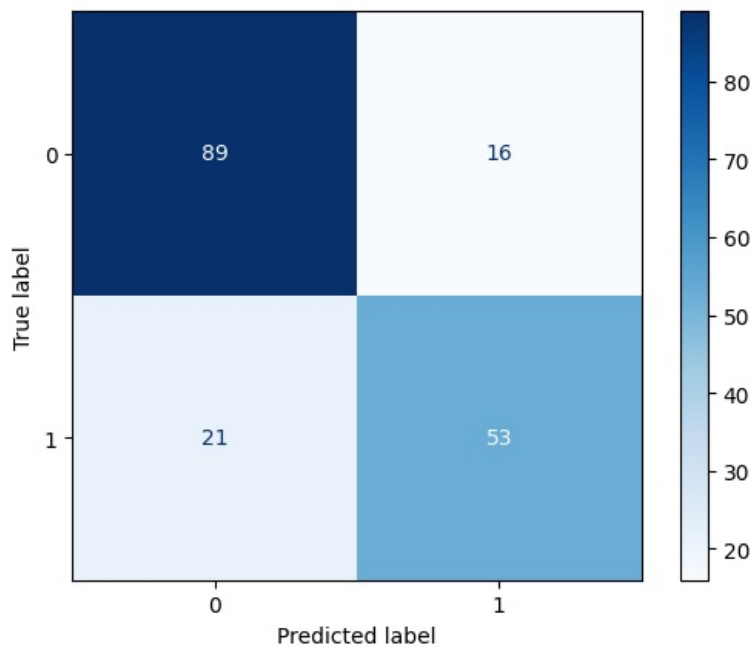
Q19. Confusion Matrix for the multivariate model

```
In [19]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)

ConfusionMatrixDisplay(confusion_matrix=cm).plot(cmap="Blues")

plt.show()
```



🚩 Section 4: Comparing the Models & Key Takeaways

Q20. Univariate vs. Multivariate accuracy

Note: We compare both models' accuracy on the same test split to see whether adding more features actually helped.

```
In [20]: print("Univariate Model Accuracy:", accuracy_score(y_test, model.predict(X_test)))

print("The multivariate model generally performs better because it uses multiple features,")
print("which provide more information than using only the fare column.")
```

Univariate Model Accuracy: 0.7932960893854749

The multivariate model generally performs better because it uses multiple features, which provide more information than using only the fare column.

📌 Key Takeaways

- **Logistic Regression** predicts probabilities using the sigmoid function, then classifies based on a 0.5 threshold.
- A **univariate model** (1 feature) is a fast baseline, but a **multivariate model** (multiple features) usually captures more of the real-world factors driving the outcome — here, `sex`, `pclass`, and `age` add context that `fare` alone cannot.
- **Data cleaning** (handling missing values, encoding categories) is a required step before any model can be trained.
- The **confusion matrix** is more informative than accuracy alone because it shows *which kind* of errors (false positives vs. false negatives) the model makes.
- The **coefficient magnitude** helps explain *why* a model predicts what it predicts — useful for interpretability, not just performance.

Notebook prepared by [Sambit7848](#) as part of a hands-on Machine Learning portfolio.